Informe U6

Informe Final — Unidad 6: Mantenimiento, Respaldo y Recuperación

Proyecto: Food Store (PostgreSQL 16) · **Integrantes:** Pamela Chirino, Lucas Agüero, Mayra Mulé

Respaldo usado: `backups/foodstore_2026-10-20_2027.dump` (`-Fc`, 1016 KB), generado el **2026-10-20 20:27** con `backup_foodstore.sh` conectado como el rol de mínimo privilegio `mant_backup` (sin contraseña en el script, vía `.pgpass`).

RTO / RPO previstos vs. observados (simulacro real, `restore_test.sh`, datos de prueba: 12 categorías / 5.002 clientes / 10.003 productos / 50.000 pedidos / 50.000 líneas):

	Previsto (SPEC_U6.md)	**Observado**
RPO	≤ 5 min (WAL continuo)	13 s (antigüedad del respaldo al correr el simulacro)
RTO	≤ 1 hora	< 1 s (`dropdb+createdb+pg_restore+validaciones`)

Los valores observados quedan muy por debajo de lo previsto porque el volumen de prueba (~115.000 filas) y la infraestructura de este banco de pruebas son chicos frente a un entorno de producción real; lo que demuestra el simulacro es que el procedimiento completo corre sin intervención manual, de punta a punta.

Resultado de las validaciones (Parte 4, sobre `foodstore_restore_test`): conteo de filas idéntico al de origen; `productos_invalidos` = 0; `detalles_invalidos` = 0. Evidencia completa en `docs/evidencia_restore_test_u6.txt`.

Problema real encontrado: `health_check.sql` usaba el literal `'food_store'` (guion bajo) en vez de `'foodstore'` (nombre real de la base). Verificado contra la base real: la consulta 1 (tamaño) rompía con `database "food_store" does not exist`, y la consulta 2 (conexiones/transacciones) no rompía pero devolvía 0 filas en silencio — 2 de los 4 controles de salud exigidos quedaban rotos o mentían sin ningún error visible. El mismo typo estaba en `backup_food_store.ps1` (variable `$Base` y nombre de archivo), y faltaba crear el rol `mant_backup` que ese script usa (referenciado en `SPEC_U6.md`, nunca creado en ningún script).

Mejora concreta: agregar un "smoke test" al inicio de cada script de mantenimiento/monitoreo — `SELECT current_database()` comparado contra una constante, con `RAISE EXCEPTION` si no coincide — para que un error de nombre de base falle de forma ruidosa en la primera corrida en vez de pasar inadvertido.

Parte asistida con IA y cómo se verificó: corrección de nombres (`health_check.sql`, `backup_foodstore.ps1`), creación y prueba real de los roles `mant_backup`/`mant_monitor`, reescritura de `maintenance.sql` para que el `ANALYZE` dependa del diagnóstico real del PASO 1 (umbral de tuplas muertas, no una lista fija de tablas), script `restore_test.sh`, y este informe. Cada cambio se verificó de forma empírica antes de aceptarlo: el bug de `health_check.sql` se reprodujo corriendo el script original contra `foodstore` real; `mant_backup` generó el `.dump` de este informe con un `pg_dump` real, y `mant_monitor` leyó `pg_stat_database` con una consulta real; `maintenance.sql` corrido contra la base real analizó solo `categoria` (62.50% de tuplas muertas reales), omitiendo el resto; y el ciclo completo de respaldo→restauración→validación se ejecutó contra PostgreSQL real, no simulado — los tiempos de este informe son los que esa corrida reportó.

No se entrega: ninguna contraseña real de PostgreSQL (las de los scripts de roles son placeholders a cambiar antes de producción; los scripts operativos usan `.pgpass`). El `.dump` se incluye por ser pequeño (1016 KB); si no entra en el aula, `docs/evidencia_restore_test_u6.txt` certifica su creación, tamaño y restauración exitosa.